1. Arquitectura de Subsistemas y Mapa de Pines (Sistemas Robóticos)

El sistema está compuesto por 6 subsistemas integrados controlados por un microcontrolador ESP32:

- **Subsistema de Control (MCU):** ESP32 NodeMCU.
- **Subsistema de Potencia:** Puente H discreto con MOSFETs y drivers BS170.
- **Subsistema de Percepción:** Sensor ultrasónico de distancia HC-SR04 (4 pines).
- **Subsistema de Límites:** Microswitches de fin de carrera (activos en bajo con pull-up interno).
- **Subsistema de Interface Visual:** Panel de LEDs indicadores de proximidad.
- **Subsistema Mecánico:** Motor CC de corriente continua con reducción para el movimiento pendular.

Mapa de Pines (GPIOs en setup.h):

- PIN_MOTOR_IN1: GPIO 18 (Control de giro Puente H)
- PIN_MOTOR_IN2: GPIO 19 (Control de giro Puente H)
- PIN_SENSOR_TRIG: GPIO 4 (Disparo del pulso ultrasónico)
- PIN_SENSOR_ECHO: GPIO 5 (Recepción del eco ultrasónico)
- PIN_FIN_CARRERA_HORARIO: GPIO 25 (Tope máximo a 180°)
- PIN_FIN_CARRERA_ANTIHORARIO: GPIO 26 (Tope mínimo a 0°)
- PIN_LED_VERDE: GPIO 21 (Zona libre / > 40 cm)
- PIN_LED_AMARILLO: GPIO 22 (Precaución / entre 15 y 40 cm)
- PIN_LED_ROJO: GPIO 23 (Alerta / < 15 cm)

2. Electrónica Analógica: Puente H Discreto y Cálculos

- **Topología:** Puente H simétrico con MOSFETs de potencia en encapsulado TO-220 (**IRF9510** Canal P arriba e **IRF510** Canal N abajo), utilizando transistores **BS170** como etapas buffer/drivers para adaptar los niveles lógicos de 3,3 V del ESP32 a los 12 V de potencia.
- **Corriente de Régimen del Motor:** Con $V_{CC} = 12V$, una resistencia interna de carga $R_m = 10\Omega$ y $R_{DS(on)} \approx 1,14\Omega$, la corriente estimada es de $I_{motor} \approx 1,08A$ (dentro del área de operación segura de los MOSFETs).
- **Protección:** Se implementaron diodos *flyback* (familia 1N4007) en paralelo con cada MOSFET para disipar de forma segura los picos inductivos generados por la bobina del motor al frenar o invertir la marcha.

3. Programación en C: Máquina de Estados Finitos (FSM)

El firmware estructurado en módulos (main.c, motor.c, sensor.c, setup.c) opera mediante un bucle infinito while(1) gobernado por una Máquina de Estados Finitos con 4 estados lógicos:

1. **ESTADO_INICIALIZACION:** Configura los registros GPIO, inicializa las estructuras de control (control_motor_t, datos_sensor_t) y posiciona el radar en 0°.

2. **ESTADO_LECTURA:** Detiene completamente el motor para evitar vibraciones mecánicas, dispara el sensor ultrasónico mediante sensor_leer_distancia(), calcula la distancia en centímetros ($d = \frac{tiempo \times 0.0343}{2}$), actualiza los LEDs de advertencia y evalúa los finales de carrera o límites de 0°/180° para decidir el próximo sentido de giro.
3. **ESTADO_BARRIDO_HORARIO / ESTADO_BARRIDO_ANTIHORARIO:** Energizan el puente H durante $150ms$, desplazan el ángulo en pasos discretos de $\pm\ 15^{\circ}$, y ceden ciclos de CPU mediante vTaskDelay para prevenir bloqueos en FreeRTOS.